**Prolog Programs**

# 17. Sum of Integers from 1 to n

Aim: To write a Prolog program to calculate the sum of integers from 1 to n.

Algorithm:

1. Read the value of n.
2. Check the base condition when n = 0.
3. Add n to the sum of integers from 1 to n−1.
4. Repeat recursively.
5. Display the final sum.
6. 


# 18. Database with Name and DOB

Aim: To write a Prolog program to create a database containing names and dates of birth.

Algorithm:

1. Define facts containing names and DOB.
2. Store the information in the database.

3. Create queries to retrieve details.
4. Search for a particular person's name.
5. Display the corresponding date of birth.
6. 


# 19. Student-Teacher-Subject-Code

Aim: To write a Prolog program to create a database for students, teachers and subject codes.

Algorithm:

1. Define student details.
2. Define teacher details.
3. Define subject codes.
4. Create relationships between them.
5. Query the database.
6. Display the matching information.

7. 


# 20. Planets Database

Aim: To write a Prolog program to create a database of planets and their properties.

Algorithm:

1. Define facts about planets.
2. Store planet names and their properties.
3. Create rules to retrieve information.
4. Execute queries for particular planets.
5. Display the results.

6. 


# 21. Towers of Hanoi

Aim: To write a Prolog program to implement the Towers of Hanoi problem.

Algorithm:

1. Initialize the number of disks and three rods.
2. If there is one disk, move it directly.
3. Move n−1 disks from source to auxiliary rod.
4. Move the largest disk to destination rod.
5. Move n−1 disks from auxiliary to destination rod.
6. Display all moves.

7. 


# 22. Check Whether a Bird Can Fly or Not

Aim: To write a Prolog program to identify whether a particular bird can fly or not using facts and rules.

Algorithm:

1. Define facts about different birds.
2. Identify flying and non-flying birds.
3. Create a rule to check flying ability.
4. Execute queries for particular birds.
5. Display whether the bird can fly or not.

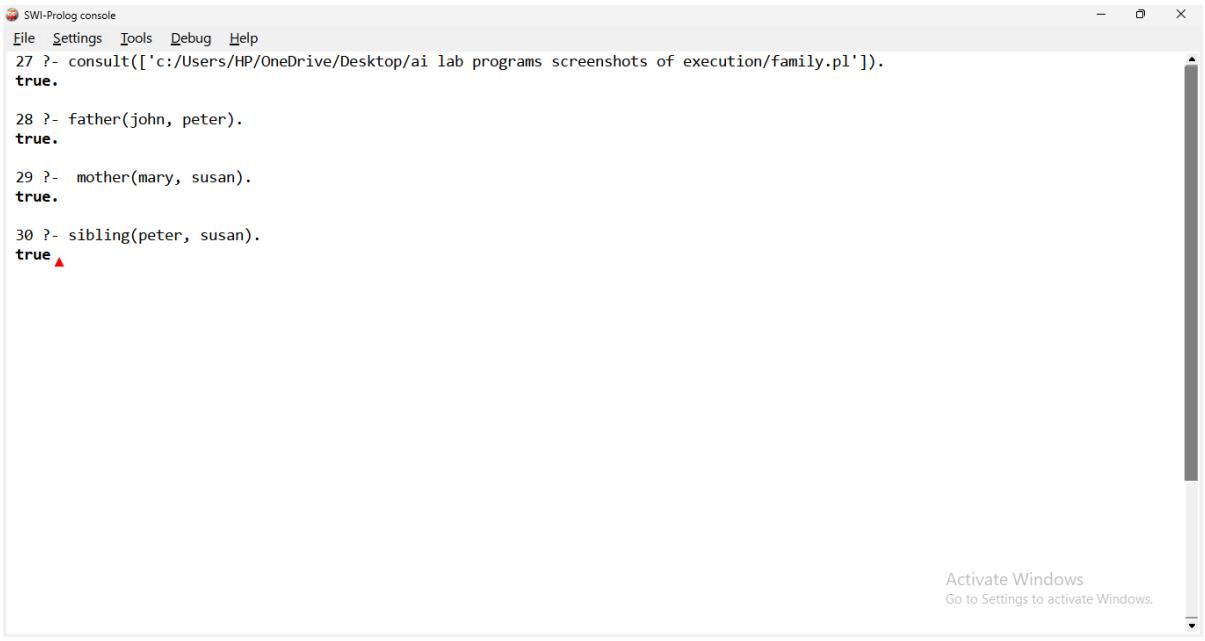
6. 


# 23. Family Tree

Aim: To write a Prolog program to implement a family tree using relationships.

Algorithm:

1. Define facts for father and mother relationships.
2. Create rules for parent relationships.
3. Define rules for siblings and grandparents.
4. Execute queries.
5. Display the family relationships.

6. 


# 24. Dieting System Based on Disease

Aim: To write a Prolog program to suggest a diet plan based on a given disease using facts and rules.

Algorithm:

1. Define diseases and sample dietary suggestions.
2. Create rules for matching diseases with suggestions.
3. Accept the disease as input.
4. Search for matching dietary suggestions.
5. Display the suggested diet.

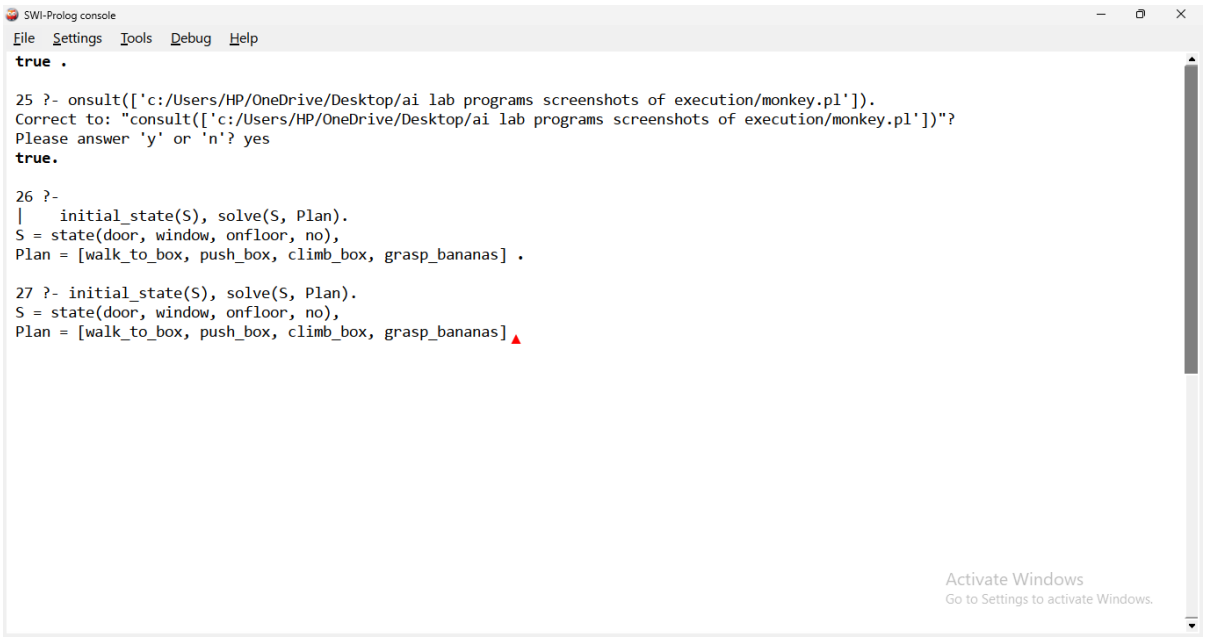
6. 


# 25. Monkey Banana Problem

Aim: To write a Prolog program to implement the Monkey Banana Problem using state-space search.

Algorithm:

1. Initialize the monkey, box and banana positions.
2. Define the initial state.
3. Identify possible actions such as move, push and climb.
4. Apply actions to reach the banana.
5. Check whether the monkey reaches the banana.
6. Display the solution steps.

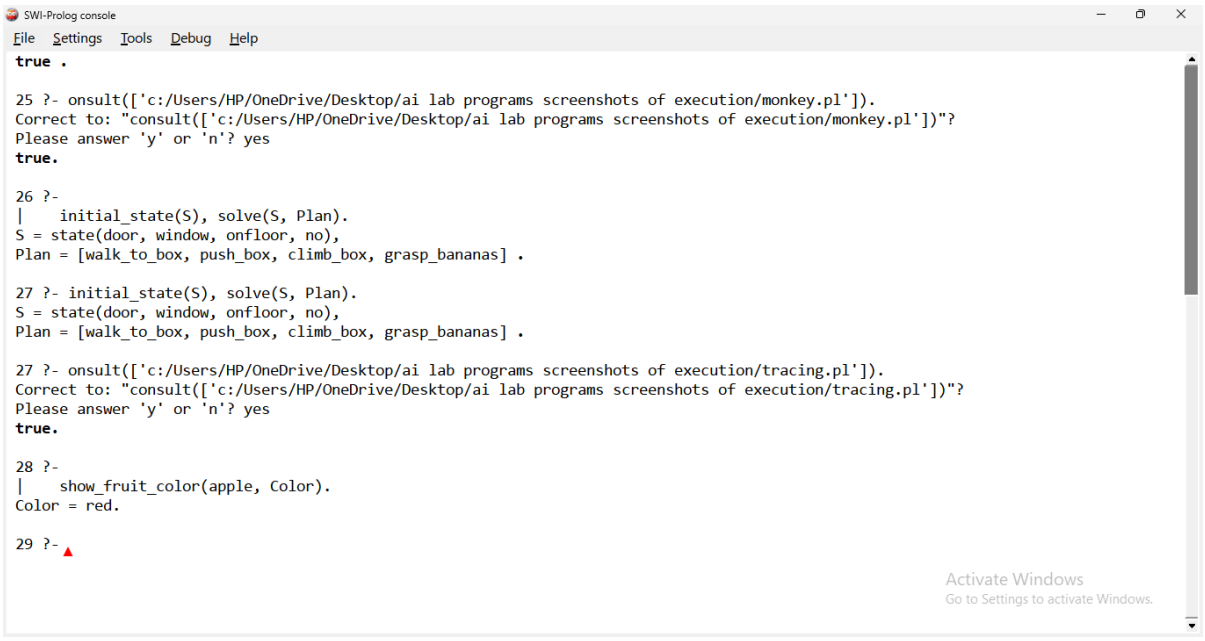
7. 


# 26. Fruit and Its Color Using Backtracking

Aim: To write a Prolog program to identify fruits and their colors using backtracking.

Algorithm:

1. Define facts for fruits and their colors.
2. Create a rule to identify fruit colors.
3. Execute a query for a fruit.
4. Search for matching facts.
5. Use backtracking to find other possible answers.
6. Display the results.

7. 


# 27. Best First Search Algorithm

Aim: To write a Prolog program to implement the Best First Search algorithm.

Algorithm:

1. Initialize the start and goal nodes.
2. Assign heuristic values to nodes.
3. Select the node with the lowest heuristic value.
4. Expand its neighbouring nodes.
5. Repeat the process until the goal is reached.
6. Display the solution path.

7. 


# 28. Medical Diagnosis

Aim: To write a Prolog program to implement a simple medical diagnosis system using symptoms and rules.

Algorithm:

1. Define facts for symptoms.
2. Create rules for possible diseases.
3. Accept symptoms as input.
4. Match symptoms with the defined rules.
5. Identify possible conditions.
6. Display the result.

7. 


# 29. Forward Chaining

Aim: To write a Prolog program to implement Forward Chaining using facts and rules.

Algorithm:

1. Define the initial facts.
2. Define the rules.
3. Check whether the conditions of a rule are satisfied.
4. Generate new facts using the satisfied rules.
5. Repeat until no new facts are generated.
6. Display the final facts.

7. 


# 30. Backward Chaining

Aim: To write a Prolog program to implement Backward Chaining using facts and rules.

Algorithm:

1. Define the initial facts.
2. Define the rules.
3. Start with a goal to be proved.
4. Check whether the goal matches an existing fact.
5. If not, check the conditions of applicable rules recursively.
6. Display whether the goal is true or false.

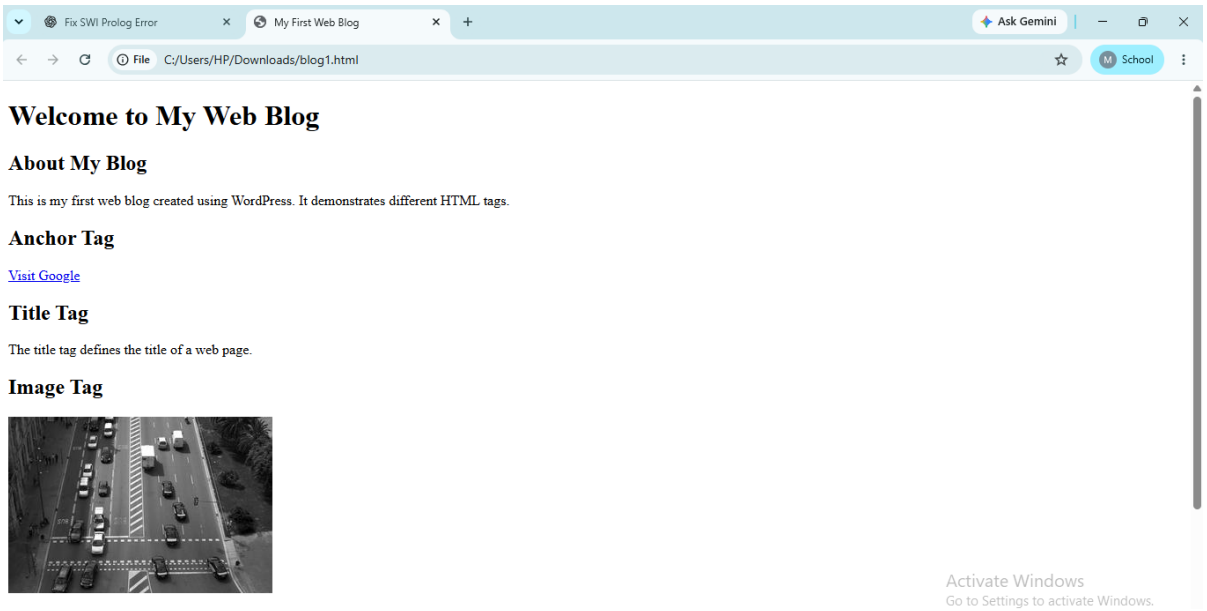
7. 


# 31. Create a Web Blog Using WordPress

Aim: To create a web blog using WordPress and demonstrate HTML tags such as Anchor Tag, Title Tag and other basic tags.

Algorithm:

1. Open the WordPress website.
2. Create an account and log in.
3. Create a new blog post.
4. Add headings, paragraphs and images.
5. Demonstrate Anchor Tag and Title Tag.
6. Publish the blog and view the output.

7. 


# 32. Pattern Matching

Aim: To write a Prolog program to implement Pattern Matching using facts, rules and variables.

Algorithm:

1. Define facts in Prolog.
2. Create rules for pattern matching.
3. Accept input patterns through queries.
4. Compare the given pattern with available facts.
5. Match the variables with corresponding values.
6. Display the matching results.

7. 


# 33. Count the Number of Vowels

Assuming this is the question you mentioned earlier.

Aim: To write a Prolog program to find the number of vowels in a given list of characters.

Algorithm:

1. Define the vowels a, e, i, o and u.
2. Read the list of characters.
3. Check each character for a vowel.
4. Increase the count if it is a vowel.
5. Repeat until the list becomes empty.
6. Display the total number of vowels.

7.


SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.
For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).
62 ?- consult(['./Users/HP/OneDrive/Desktop/ai lab programs screenshots of execution/class.pl']
true.
63 ?- match_student(ravi, cse).
true.
64 ?- match_student(raju, ece).
true.
65 ?- match_student(rahul, cse).
true.
66 ?- consult(['c:/Users/HP/OneDrive/Desktop/ai lab programs screenshots of execution/vowel.pl']).
true.
67 ?- count_vowels([a,e,i,o,u], N).
N = 5 .
68 ?- count_vowels([h,e,l,l,o], N).
N = 2
Activate Windows
Go to Settings to activate Windows.